

AI 기반 자동화 MVP 빌더

프로젝트 중간 발표

Recap: 우리 서비스가 만드는 것

MVP-Builder — 자연어 요구사항 → 실행 가능한 코드베이스 자동 생성

풀고자 하는 문제

기존 AI 빌더(Lovable, Bolt.new): 토큰 사용량 불투명 + 코드 소유권 없음 + 블랙박스 생성

핵심 플로우

요구사항 입력 → ERD/API 스펙 생성 → 사용자 검토·확정 → 코드 자동 생성 → 내 GitHub

경쟁사 대비 차별점 3가지

- 코드 전에 분석 문서 먼저 — 사용자가 제어
- docker compose up 즉시 실행 보장
- 사용자 Claude API Key 직접 사용 — 플랫폼 과금 없음

타겟: 개발자 및 기술 지향 창업가를 위한 솔루션

Tech Stack

Frontend

Next.js 14 (App Router)
TypeScript
Tailwind CSS
Zustand

Backend

NestJS
TypeScript
Prisma ORM

AI Layer

@anthropic-ai/sdk
tool use 기반 에이전트
직접 정의·오케스트레이션

Queue

BullMQ + Redis
파이프라인 비동기 처리
상태 영속성 유지

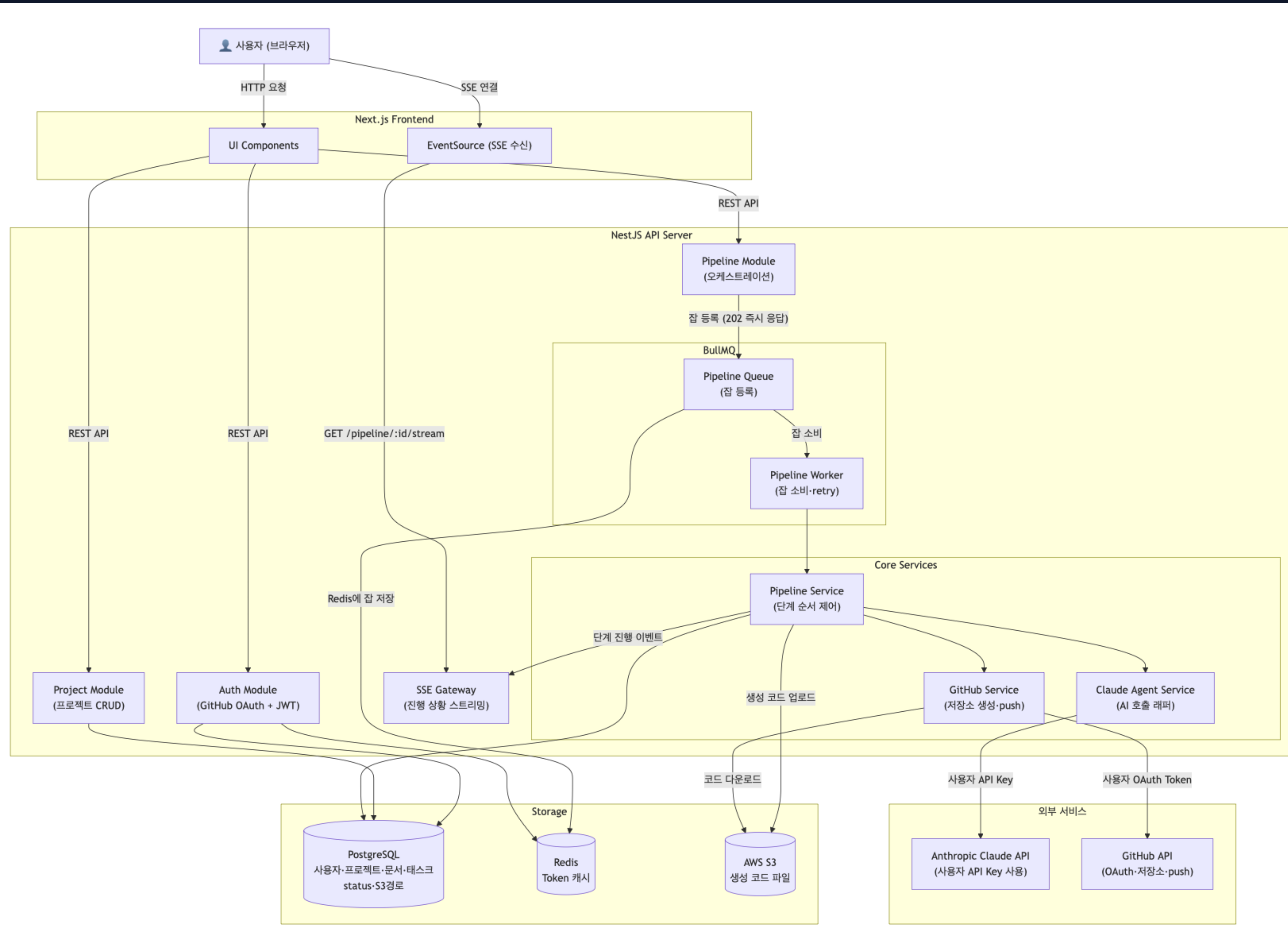
Database / Storage

PostgreSQL
AWS S3 (생성 코드 파일)

Infra

Docker Compose
AWS EC2
GitHub Actions

시스템 아키텍처



시스템 아키텍처 흐름

⚡ 비동기 파이프라인 처리

사용자 요청은 **NestJS API**가 받고, 수 분짜리 파이프라인 작업은 **BullMQ Queue**에 등록 후 **202 즉시 응답**

🤖 AI 에이전트 단계별 실행

Pipeline Worker가 Queue에서 잡을 꺼내 **Phase 1→2→3** 순서로 **Claude Agent**를 단계별로 호출 — 각 Phase는 정의된 Tool 집합으로 구조화된 출력을 생성

🛡️ 보안 격리 스토리지

생성 코드는 **S3**에만 저장, 플랫폼 서버에서 실행하지 않음 (보안 격리)

주요 로직: #1 Agent Loop (Phase 1 & 2)

각 Phase마다 Claude에게 Tool을 정의해두고, Claude가 Tool을 호출하는 방식으로 구조화된 출력을 강제. Claude가 자유 텍스트 대신 Tool Input(JSON)으로 결과를 반환 → 파싱 없이 바로 DB/S3에 저장

Phase 1 — 분석 문서 생성 (runAgentLoop)

Tool	역할
design_erd	요구사항 → Mermaid ERD 생성
design_api_spec	RESTful API 스펙 마크다운 생성
design_architecture	시스템 아키텍처 마크다운 생성
design_directory_structure	기술 스택에 맞는 파일 경로 목록 JSON 생성

→ 4개 Tool을 순서대로 호출하는 Agent Loop. 완료 후 AnalysisDocument DB 저장 + 디자인 시스템 1회 생성

Phase 2 — 태스크 분해 (runWithTool 단건)

Tool	역할
generate_tasks	확정 문서 → [{name, description, type, order_index}] 태스크 배열 생성

→ Tool 1개, 단건 호출. type(BACKEND/FRONTEND)으로 Phase 3 분기 결정

주요 로직: #1 Agent Loop (Phase 3)

Task type(BACKEND/FRONTEND)에 따라 분기 처리되며, 각 태스크별로 지정된 Tool을 통해 코드를 자동 생성합니다.

Phase 3 — 코드 생성 (태스크별 반복)

Tool	대상	방식
generate_test_code	BACKEND	runAgentLoop — 테스트 코드 먼저
generate_implementation_code	BACKEND	runAgentLoop — 테스트 통과하는 구현 코드
generate_ui_component	FRONTEND	runWithTool 단건 — 디자인 시스템 주입 후 컴포넌트 생성

→ 생성된 파일은 S3 업로드 후 Task status DONE 갱신

주요 로직: #2 BullMQ 비동기 메시지큐

🔗 파이프라인 흐름

POST /pipeline/start

- PipelineRun DB 생성 (status: RUNNING)
- BullMQ Queue에 잡 등록
- 202 즉시 응답

Pipeline Worker (별도 프로세스)

- Queue에서 잡 소비
- Phase 1 실행 → 완료 시 문서 검토 요청

POST /pipeline/confirm

- Phase 2 → Phase 3 순서 실행
- 각 단계 완료 시 SSE로 실시간 이벤트 전송

💡 도입 이유

- Phase 3 코드 생성은 태스크 수에 따라 수 분 소요 → HTTP 타임아웃 (30초) 초과 불가피
- BullMQ로 잡을 Queue에 등록하면 API는 202 즉시 응답, Worker가 백그라운드에서 처리

🔄 상태 관리 및 Resume 전략

- **PipelineRun**: RUNNING / COMPLETED / FAILED
- **Task**: PENDING → IN_PROGRESS → DONE / FAILED
- Phase 3 실패·재시작 시 status = DONE인 태스크는 **skip**
- 실패한 태스크부터 재실행 → 전체 재생성 비용 방지

현재 진행도

Sprint 1 (파이프라인 핵심)

태스크	상태
모노레포 + Docker Compose + Prisma 스키마	✓
BullMQ Pipeline Worker	✓
Claude Agent SDK 래퍼 (runAgentLoop / runWithTool / stream)	✓
Phase 1 — 분석 문서 생성 + 디자인 시스템	✓
Phase 2 — 태스크 분해 (BACKEND/FRONTEND 분류)	✓
Phase 3 — 백엔드 TDD + 프론트엔드 컴포넌트 생성 + S3 업로드	✓
SSE 실시간 스트리밍 Gateway	🔨

Sprint 2 (인증·연동·프론트·배포)

태스크	상태
PipelineService 상태 머신 (Phase 전환·resume)	☐
GitHub Service (저장소 생성 + S3 코드 push)	☐
GitHub OAuth + JWT 인증	☐
사용자별 Claude API Key 암호화 저장·사용	☐
프론트엔드 UI (Next.js)	☐
Phase 3 샌드박스 테스트 실행·재생성 루프	☐
EC2 배포 + GitHub Actions CI/CD	☐

코드 생성 검증: 현재 방식 & 다음 목표

❗ 현재 한계 및 방식

- Phase 3에서 생성된 코드는 S3에 업로드되고 끝
- 생성된 코드가 실제로 빌드·테스트를 통과하는지 검증하지 못하고 있음
- 현재는 Claude가 올바른 코드를 생성했다고 신뢰하는 방식

현재 검증 방식

Phase 3 서비스 자체에 대한 Unit Test 28개 작성·통과.

"어떤 Tool을 올바른 순서로 호출했는가", "S3에 올바른 인자로 업로드했는가"를 검증. 실제 생성 코드의 동작 여부는 미검증.

🎯 다음 목표 — Docker 샌드박스 테스트 루프

1. S3에서 생성 코드 다운로드
2. 임시 Docker 컨테이너 실행
3. `npm install && npx jest`
4. 테스트 통과 시 Task DONE 확정
5. 테스트 실패 시 에러 로그를 Claude에 전달
6. 해당 파일 재생성 → 재실행 (최대 3회)
7. 3회 초과 시 Task FAILED 처리

감사합니다.